

Christian A. Wilkins

September 23, 2026

Dr. KMA Solaiman

CMSC 471 Introduction to A.I.

1. Heuristic Design and Admissibility

a. Steps

- i. $h(n) = \sum$ mismatched characters between n and goal
 1. `wrong = 0`
 2. `for index in len(state):`
 3. `if state [index] != goal [index]:`
 4. `wrong += 1`
 5. `return wrong`
- ii. The unavoidable remaining cost the heuristic lower-bounds is the number of remaining actions to reach the goal. It looks at every index that does not match the goal word and sums the count of those actions to translate the word.
- iii. This function cannot overestimate the true optimal cost because every action costs exactly 1 which means that the true path cost is at a minimum of the number of mismatched characters. At the end there is no mismatched characters so the function returns 0. As such the function is admissible because it will always be less than or equal to the true cost.

b. Scrabble

- i. $h(n) = \sum$ scrabble values of each of the goal letters at the mismatched indexes
 1. `cost_total = 0`
 2. `for index in len(state):`
 3. `if state[index] != goal [index]:`
 4. `letter = goal [index]`
 5. `if letter in "aeioulnstr":`
 6. `cost_total += 1`
 7. `elif letter in "dg":`
 8. `cost_total += 2`
 9. `elif letter in "bcmp":`

```

10.         cost_total += 3
11.     elif letter in "fhvwy":
12.         cost_total += 4
13.     elif letter in "k":
14.         cost_total += 5
15.     elif letter in "jx":
16.         cost_total += 6
17.     elif letter in "qz":
18.         cost_total += 10
19.     return cost_total

```

- ii. The unavoidable remaining cost the heuristic lower-bounds is the sum of cost of each scrabble value of the mismatched indexes. Each scrabble value corresponds to the value of a goal character in the goal statement and at a minimum the value of each mismatched goal character must be paid to turn the initial into the goal.
- iii. This function cannot overestimate the true optimal cost because the true cost to reach the goal does not count all intermediate steps but rather only the necessary ones to reach the goal. This means that the cost will be equal to the scrabble values of all of the characters in the goal word that were not initially in the starting word which is what the heuristic function calculates.

c. Frequency

- i. $h(n) = \sum \text{mismatched characters between } n \text{ and goal}$
 - 1. wrong = 0
 - 2. for index in len(state):
 - 3. if state [index] != goal [index]:
 - 4. wrong += 1
 - 5. return wrong
- ii. The unavoidable remaining cost the heuristic lower-bounds is the number of remaining actions to reach the goal. It looks at every index that does not match the goal word and sums the count of those actions to translate the word.
- iii. This function cannot overestimate the true optimal cost because the frequency function requires that new letters cost 1 + their rarity and because the rarities are positive numbers, each action costs at least 1 + 0. As such the function is admissible because it will always be less than the true cost.

2. Does the Heuristic actually help?

Word Pair	Cost Measure	UCS Cost	UCS Nodes Exp.	A* Cost	A* Nodes Exp.	Reduction
Why->Ask	Steps	5	773	5	16	97.93%
Why->Ask	Scrabble	9	765	9	13	98.3%
Why->Ask	Frequency	38.855	808	38.855	755	6.56%
Bear->Duck	Steps	4	691	4	6	99.13%
Bear->Duck	Scrabble	11	2981	11	6	99.79%
Bear->Duck	Frequency	38.24	1084	38.24	720	33.58%

1. Yes, UCS and A* always returned the same cost
 2. A* produced the largest percentage reduction in expanded nodes using the Scrabble cost measure for both Why->Ask and Bear->Duck.
 3. For both word pairs, the heuristic was least informative for the Frequency cost measure.
 4. The differences between the performances of the heuristics is based on how closely they follow the true remaining cost. For Steps, the action had a cost of one and the heuristic was measuring how many actions were needed to change the word into the goal. This meant that the number of remaining actions would always equal the heuristic cost so it was very informative for the algorithm and allowed it to expand much fewer nodes than UCS. For Scrabble, it considered every incorrect index and the true scrabble cost to correct that index which meant that the heuristic was closely related to the true cost. For Frequency, the heuristic considers mismatched steps which always keeps it admissible however not necessarily close to the true cost. This means that the heuristic was always significantly underestimating the true cost and was least informative to the algorithm meaning it still expanded many nodes.
3. Reversibility and direction-dependent costs
- a. Steps:
 - i. Yes
 - ii. Mismatched # of indexes will always be the same whether forward or backward
 - b. Scrabble:
 - i. No
 - ii. If the initial word has many low value characters but the goal has high value characters, then the cost to transform one way or the other will be different
 - iii. Hero, Zero
 dc(hero,zero,scrabble) cost:10.00; time:0.000; solution:hero zero;
 deltas:[0.0]

dc(zero,hero,scrabble) cost:4.00; time:0.000; solution:zero hero;
deltas:[0.0]

c. Frequency

i. No

ii. The cost is based on the rarity of the goal word independent of the initial word so if a goal word has a higher rarity than a start word, it would not be the same cost to transform back and forth.

iii. Hate, Love

dc(hate,love,frequency) cost:19.16; time:0.038; solution:hate have
have love; deltas:[-16.163, -15.572, -3.869]

dc(love,hate,frequency) cost:21.59; time:0.087; solution:love have
have hate; deltas:[-18.593, -6.89, -6.3]

4. LLM Heuristic Audit

a. Part 1:

i. ChatGPT 5.6 Sol.

ii. Prompt: “In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.”

iii. Heuristic:

Let g be the goal word and let

$$d(n) = h_0(n)$$

be the Hamming distance from state n to g .

For each distance layer k , define the minimum rarity of any legal dictionary word that is exactly k letters away from the goal:

$$\rho_k = \min_{\substack{w \in D \\ h_0(w)=k}} \text{rarity}(w).$$

Then use the heuristic

$$h_R(n) = d(n) + \sum_{k=0}^{d(n)-1} \rho_k$$

or equivalently,

$$h_R(n) = \sum_{k=0}^{d(n)-1} (1 + \rho_k).$$

- b. $h_r(n) = d(n) + \sum_{k=0}^{d(n)-1} \rho_k$
- c. Any path d letters from the goal must pass through states $d-1, d-2, \dots, 1$, and 0 letters away from goal. A word at distance k is at least $1 +$ the minimum rarity of k or ρ_k . Therefore, the cost will always be at a minimum, $1 +$ the minimum rarity meaning the function is always admissible.
- d. N/A, the system makes sense logically.
- e.

Problem	Heuristic	Cost	Nodes Expanded
Why->Ask	H0	38.85	755
Why->Ask	New	38.85	294
Bear->Duck	H0	38.24	720
Bear->Duck	New	38.24	75

The evidence, although limited, supports using the new heuristic as there are significantly less nodes expended for both examples (Why->Ask 61.05% and Bear->Duck 89.58%).

Collaboration, sources, and generative AI:

1. ChatGPT 5.6 Sol
2. Question 4.5 for understanding and implementing its heuristic in code.
3. Went step by step with the model to understand each line of new heuristic code, how it translated to the equation, and edit to be correct.